Product Requirements Document

EnergyIQ: Smart Energy Consumption Optimizer

1. Executive Summary

Vision: EnergyIQ transforms raw smart-meter telemetry into a proactive, intelligent virtual energy manager. By isolating true structural anomalies from normal baseline variance and exposing invisible "ghost loads," the platform empowers facility managers to dramatically reduce carbon footprints and operational costs without requiring deep data science expertise.

Target Users: Facility Managers, Building Administrators, Corporate ESG Officers, and University Campus Operations.

Hackathon Strategy: Execute a flawless, offline-capable ML data pipeline using the provided UCI Tetouan City dataset, eliminating live API risk while delivering high-impact, interactive visualizations and GenAI-powered sustainability roadmaps.

2. 100% Free Technology Stack

Layer	Technology	Purpose / Role
Frontend / UI	Streamlit (Python)	Rapid, interactive web application framework. Requires no HTML/JS.
Data Processing	Pandas & NumPy	CSV ingestion, time-series resampling, and mathematical grouping.
Machine Learning	Scikit-Learn (Isolation Forest)	Statistical anomaly detection to separate normal spikes from true waste.
Visualizations	Plotly & Seaborn	Interactive line charts, ghost-load heatmaps, and green grid dials.
GenAI Engine	Groq Cloud API (Llama 3 70B)	Free, ultra-fast LLM inference for natural-language mitigation reports.

3. System Architecture & Data Flow



4. User Journey (Demo Flow)

- **The Hook (Upload):** The user drops the `tetouan_power.csv` file into the UI. The dashboard populates instantly.
- **The Baseline (Timeline):** The user explores an interactive line chart of historical energy consumption, visually noting the grey-shaded "Peak Load" hours.
- **The Reveal (Anomaly Toggle):** The user clicks "Detect Anomalies." The ML model overlays bright red dots on the chart, mathematically proving which spikes are true equipment malfunctions rather than normal usage.
- **The Evidence (Heatmap):** The user scrolls to the "Ghost Load" heatmap, immediately seeing glowing red squares at 3:00 AM on Sundays—proving massive off-hour energy waste.
- **The Value (ROI Simulator):** The user drags a "Shift Operations" slider. A digital scoreboard updates instantly, calculating dollars saved and carbon reduced by avoiding peak-pricing hours.
- **The Action (AI Report):** The user clicks "Generate Plan." The Groq AI outputs a formal email outlining exactly which machines to shut off and when, based purely on the data.

5. Feature Specifications (Core Requirements)

Feature 1: Data Ingestion & Time-Series Parsing

Description: Secure, local ingestion of smart meter telemetry data.

Code Requirements: Use `st.file_uploader` to accept a CSV. Use `pd.read_csv()` to load the dataframe. Crucially, cast the timestamp column to datetime objects (`pd.to_datetime()`) and set it as the index. Handle missing values using forward-fill (`df.ffill()`) to ensure the ML models do not crash during the live demo.

Feature 2: Dynamic Baseline & Peak-Load Dashboard

Description: An interactive line chart displaying total energy consumption over time, with automatic highlighting of top decile usage.

Code Requirements: Calculate the 90th percentile of energy usage using `df['consumption'].quantile(0.90)`. Use `plotly.graph_objects` to draw a line chart. Use `add_vrect()` or colored markers in Plotly to shade regions where consumption exceeds the 90th percentile threshold, visually representing "Peak Hours."

Feature 3: Contextual Anomaly Engine

Description: Machine learning module that identifies statistically abnormal energy draw, accounting for time of day.

Code Requirements: Import `IsolationForest` from Scikit-Learn. Extract temporal features (hour, day of week) from the datetime index. Train the model on `['consumption', 'hour']` with a contamination rate of 0.05. Predict outliers (-1). Filter the dataframe for these outliers and overlay them as a red scatter plot trace on top of the main Plotly line chart.

6. Feature Specifications (Custom Innovations)

Innovation 1: The "Ghost Load" Heatmap

Description: Exposes energy wasted when the facility is closed by mapping average usage to a 7x24 grid.

Code Requirements: Create a pivot table using Pandas: `pd.pivot_table(df, values='consumption', index=df.index.dayofweek, columns=df.index.hour, aggfunc='mean')`. Map the index to day names (Mon-Sun). Render this matrix using `plotly.express.imshow()` with a continuous color scale (e.g., 'Reds' or 'Inferno').

Innovation 2: "What-If" ROI & Carbon Simulator

Description: Interactive slider demonstrating the financial and environmental impact of shifting loads away from peak hours.

Code Requirements: Implement a Streamlit slider: `shift_hours = st.slider("Shift Load", 0, 6)`. Create a copy of the dataframe and use `df['consumption'].shift(shift_hours)`. Calculate simulated cost: assign a high rate (e.g., $0.15/kWh) to peak hours (14:00-18:00) and a low rate ($0.08/kWh) elsewhere. Display the delta (original cost vs. simulated cost) using `st.metric()`.

Innovation 3: Green Sync Grid Matcher

Description: Recommends the optimal time to consume heavy power based on grid renewable percentages.

Code Requirements: Generate a mock sine-wave distribution representing "Solar Grid Availability" peaking at 12:00 PM. Multiply the facility's shifted energy curve by this availability score. Display a Plotly Indicator (Gauge Chart) showing the "Green Sync Score" (0-100%).

Innovation 4: GenAI Actionable Mitigation Plan

Description: Translates detected anomalies into a C-suite ready email report.

Code Requirements: Use the `groq` Python SDK. Format a strict prompt containing summary statistics: total anomalies, highest ghost load hour, and simulated ROI savings. Ask Llama 3 to

7. Code Generation Prompt Strategy

When you are ready to write the code, do not ask the AI to build the entire app at once. It will make mistakes. Copy and paste these exact prompts into the AI sequentially to build `app.py`:

- **Prompt 1 (Setup & Data):** "Write a Streamlit app using Python and Pandas. Add a file uploader. When a CSV is uploaded, parse a 'Datetime' column, set it as the index, and display a basic dataframe summary."
- **Prompt 2 (Main Chart):** "Update the code. Add a Plotly line chart showing 'Zone 1 Power Consumption' over time. Calculate the 90th percentile and shade those peak hours on the Plotly chart."
- **Prompt 3 (ML Engine):** "Update the code. Add an IsolationForest model from Scikit-Learn. Train it on the consumption column and hour of the day. Find the anomalies (-1) and plot them as red scatter dots on the existing Plotly chart."
- **Prompt 4 (Heatmap):** "Update the code. Create a Pandas pivot table for average consumption by Day of Week vs. Hour of Day. Plot this as a Plotly Heatmap labeled 'Ghost Load Analysis'."
- **Prompt 5 (ROI Simulator):** "Add a Streamlit slider (0-6 hours). When shifted, shift the consumption column down by X hours. Calculate the cost difference assuming peak hours cost $0.20 and off-peak cost $0.08. Display the savings using st.metric."
- **Prompt 6 (Groq LLM):** "Add an integration with the Groq API. Take the total number of anomalies and the ROI savings variable, pass them into a prompt to Llama 3 70B, and display the generated executive summary text in the app."

Pre-Hackathon Checklist:

1. Install Python and VS Code.
2. Run `pip install streamlit pandas plotly scikit-learn groq`.
3. Download the Tetouan City CSV dataset.
4. Generate your free Groq API key.